BÁO CÁO RÀ SOÁT KỸ THUẬT TOÀN DIỆN (CODE AUDIT)

Hệ thống IPGS Remote Control (CCU ↔ ZCU)

Mã tài liệu: `REVIEW-remote-control-full-audit-2026-10-31`

Ngày lập: 2026-10-31

Người thực hiện: Senior Developer & Tech Lead

Phạm vi: Toàn bộ codebase (`IPGS.RemoteControl.ZcuAgent`, `IPGS.RemoteControl.CcuClient`, `IPGS.RemoteControl.CcuUI`) và các commit tính năng mới trên nhánh `feature/remote-improve-rustdesk`.

Mức độ ưu tiên: P1 (Chuẩn bị phát hành Release 1.2 ổn định cao)

---

1. TỔNG QUAN HIỆN TRẠNG HỆ THỐNG

Hệ thống Remote Control phục vụ việc điều khiển từ xa các máy trạm kiosk/cổng kiểm soát ZCU (Linux X11) từ máy trạm trung tâm CCU (Windows/Avalonia 12). Hệ thống bao gồm 3 phân hệ chính:

1. **`IPGS.RemoteControl.ZcuAgent`** (Daemon Linux, net8.0-linux-x64): Chạy ngầm systemd, capture màn hình qua X11 MIT-SHM/XGetImage, mã hóa JPEG bằng SkiaSharp, nhận lệnh input qua XTest/X11, hỗ trợ truyền file nội bộ, đồng bộ clipboard và giám sát phần cứng.
2. **`IPGS.RemoteControl.CcuClient`** (Class Library, net8.0): Quản lý giao thức TCP/TLS, mã hóa gói tin nhị phân, heartbeat PING/PONG, luồng tự động kết nối lại (reconnect with backoff), các dịch vụ quản trị từ xa qua SSH (cài đặt ứng dụng, quản lý kiosk, mount CIFS/NFS).
3. **`IPGS.RemoteControl.CcuUI`** (Desktop App, Avalonia 12 / .NET 8): Ứng dụng quản trị giao diện người dùng, hiển thị luồng màn hình thời gian thực, quản lý đa kết nối (MultiRemote), truyền file hai chiều, dán trực tiếp file/thư mục qua phím tắt Ctrl+V, và theo dõi sức khỏe phần cứng (CPU/RAM/GPU/NPU).

---

2. KẾT QUẢ KHẢO SÁT & CÁC VẤN ĐỀ KỸ THUẬT PHÁT HIỆN

Qua quá trình rà soát chi tiết từng dòng lệnh, luồng dữ liệu mạng, bộ nhớ và các kịch bản ngoại lệ, nhóm kỹ thuật ghi nhận các phát hiện theo từng nhóm rủi ro:

2.1 Hiệu năng & Rò rỉ Bộ nhớ (Memory & GC Pressure) — Mức độ: NGHIÊM TRỌNG

Vấn đề M1: Phình Large Object Heap (LOH) ~124 MB/s tại RemoteScreenViewModel

- **Vị trí:** `IPGS.RemoteControl.CcuUI/ViewModels/RemoteScreenViewModel.cs` (Dòng 394–410)
- **Thực trạng:**

Trong hàm `OnFrameReceived`, với mỗi frame JPEG nhận được từ mạng TCP (tốc độ 15 fps ở độ phân giải 1080p):

```
var wb = new WriteableBitmap(new Avalonia.PixelSize(w, h), ...);
byte[] pixData = new byte[toRender.ByteCount]; (1920 x 1080 x 4 = 8,294,400 bytes, xấp xỉ 8.3 MB)
Marshal.Copy(pixPtr, pixData, 0, pixData.Length);
using (var locked = wb.Lock()) Marshal.Copy(pixData, 0, locked.Address, pixData.Length);
```

- **Hệ quả:**

1. Mọi mảng byte có kích thước ≥ 85,000 bytes được runtime .NET cấp phát trực tiếp trên **Large Object Heap (LOH)**. Tại 15 fps, ứng dụng cấp phát **124.4 MB/giây** rác LOH.
2. LOH chỉ được thu gom khi kích hoạt Gen 2 GC (Full GC), gây hiện tượng giật cục (micro-stuttering) và phình RAM tiến trình CCU lên hàng GB chỉ sau vài phút streaming.
3. Cơ chế sao chép 2 lần trung gian (Unmanaged Skia Pixels → Managed `byte[]` LOH → Unmanaged `locked.Address`) gây lãng phí băng thông CPU và bộ nhớ đệm cache L2/L3.

Vấn đề M2: Tái tạo SKBitmap unmanaged liên tục trên JpegEncoder

- **Vị trí:** `IPGS.RemoteControl.ZcuAgent/Capture/JpegEncoder.cs` (Dòng 43–65)
- **Thực trạng:**

Mỗi lần gọi `EncodeJpeg(frame, quality)`, một đối tượng `new SKBitmap(info)` mới được tạo ra và giải phóng. Ở độ phân giải 1080p, mỗi frame SkiaSharp cấp phát ~8.3 MB bộ nhớ unmanaged thông qua `glibc malloc`.

- **Hệ quả:**

Theo ghi nhận tại `GOTCHAS.md` (G002), trên môi trường Linux (đặc biệt là các bo mạch ARM64 hoặc Intel Atom), glibc malloc arena khi giải phóng không trả ngay bộ nhớ cho hệ điều hành nếu không kích hoạt `malloc_trim(0)`. Việc cấp phát và hủy 15 lần/giây dẫn đến hiện tượng phân mảnh heap unmanaged và phình VmRSS nhanh chóng.

Vấn đề M3: Cấp phát mảng 64 KB liên tục khi upload file

- **Vị trí:** `IPGS.RemoteControl.CcuClient/RemoteControlClient.cs` (Dòng 227)
- **Thực trạng:**

```
await SendLockedAsync(MessageType.FileChunk, buf.AsSpan(0, 1 + n).ToArray(), ct);
```

Gọi `.ToArray()` trong vòng lặp đọc file khiến mỗi chunk 64 KB sinh ra một mảng mới trên heap. Truyền file 200 MB sinh ra ~3,200 allocations không cần thiết.

2.2 Luồng xử lý Bất đồng bộ & Đồng bộ hóa (Concurrency & Threading) — Mức độ: CAO

Vấn đề C1: Áp lực ThreadPool và tạo tác vụ thừa tại RunInputInjectorLoopAsync

- **Vị trí:** `IPGS.RemoteControl.ZcuAgent/Net/ClientSession.cs` (Dòng 614–627)
- **Thực trạng:**

`RunInputInjectorLoopAsync` vốn dĩ đã là một consumer loop bất đồng bộ chạy trên một luồng nền riêng biệt đọc từ `_inputQueue`. Tuy nhiên, với mỗi sự kiện di chuột hoặc gõ phím, code lại thực hiện:

```
var task = Task.Run(action, ct);
if (await Task.WhenAny(task, Task.Delay(InjectWarnMs, ct)) != task)
```

- **Hệ quả:**

Khi người dùng di chuột liên tục (60 sự kiện/giây), mỗi sự kiện lại đẩy thêm một `Task.Run` vào .NET ThreadPool, tạo một `Task.Delay` hẹn giờ và một `Task.WhenAny`. Tổng cộng sinh ra hơn 180 Task allocations/giây và hàng loạt timer callbacks, làm nghẽn ThreadPool của các CPU yếu (Intel Celeron J3xxx, Rockchip RV1126).

Vấn đề C2: Cập nhật UI từ ThreadPool trong RemoteScreenControl.PasteAsync

- **Vị trí:** `IPGS.RemoteControl.CcuUI/Views/RemoteScreenControl.axaml.cs` (Dòng 274)
- **Thực trạng:**

```
_ = Task.Delay(6000).ContinueWith(_ => PasteStatus?.Invoke(null));
```

`Task.Delay.ContinueWith` kích hoạt callback trên luồng ThreadPool. Event `PasteStatus` được lắng nghe bởi cửa sổ `RemoteScreenWindow` để cập nhật thanh tiêu đề (`Title = ...`). Việc ghi trực tiếp vào UI control từ luồng không phải UI Thread (không bọc qua `Dispatcher.UIThread.Post`) vi phạm nguyên tắc luồng của Avalonia và có thể gây văng app ngẫu nhiên trên Windows/Linux.

Vấn đề C3: Unbounded Delegate Allocations trên _inputQueue

- **Vị trí:** `IPGS.RemoteControl.ZcuAgent/Net/ClientSession.cs` (Dòng 86, 434, 448)
- **Thực trạng:**

`_inputQueue` là `Channel.CreateUnbounded<Action>()`. Mỗi lần nhận `MouseMove`, code cấp phát một closure delegate mới:

```
_inputQueue.Writer.TryWrite(() => _injector.Move(mx, my));
```

Không có cơ chế coalescing (gộp tọa độ di chuột thừa). Nếu X server bị khựng 0.5s, hàng trăm delegate di chuột cũ vẫn nằm xếp hàng và được thực thi lần lượt khiến con trỏ chuột bị trôi (lag chuột).

2.3 Bảo mật & An toàn Hệ thống (Security & Compliance) — Mức độ: CAO

Vấn đề S1: Lỗ hổng bảo mật nghiêm trọng (High Severity) trong thư viện SSH.NET

- **Vị trí:** Toàn bộ các project sử dụng package `SSH.NET` phiên bản `2025.1.0`.
- **Thực trạng:**

Compiler và dotnet restore cảnh báo trực tiếp:

  - `NU1903: Package 'SSH.NET' 2025.1.0 has a known high severity vulnerability: GHSA-mggc-4xg6-vcxf`
  - `NU1903: Package 'SSH.NET' 2025.1.0 has a known high severity vulnerability: GHSA-q939-rpr3-3284`

- **Hệ quả:**

Rủi ro bị tấn công kênh truyền SSH hoặc từ chối dịch vụ (DoS). Cần nâng cấp package lên bản vá chính thức mới nhất.

Vấn đề S2: Lộ mật khẩu sudo qua Biến môi trường (/proc/<pid>/environ)

- **Vị trí:** `IPGS.RemoteControl.CcuClient/RemoteAppInstallService.cs` (Dòng 67) & `ZcuRemoteInstallerService.cs`
- **Thực trạng:**

```
string envCmd = $"env DISPLAY=:0 DBUS_SESSION_BUS_ADDRESS=...
KIOSK_SUDO_PASS={ShellQuote.Quote(options.SudoPassword)}";
RunCommand(ssh, "{envCmd} bash -c 'echo \"KIOSK_SUDO_PASS\" | sudo -S dpkg -i ...'",
Log);
```

- **Hệ quả:**

Biến môi trường của tiến trình hiển thị công khai trong `/proc/<pid>/environ` cho các tiến trình khác cùng chạy dưới quyền user đọc được, hoặc khi ai đó kiểm tra bằng lệnh `ps e`. Trong khi đó, module `RemoteCommandWindow` và `BulkActionWindow` đã tuân thủ chuẩn an toàn khi đẩy mật khẩu qua STDIN stream (`sudo -S -p ''`). Cần thống nhất cơ chế truyền qua STDIN channel cho toàn bộ các service.

Vấn đề S3: Cơ chế chứng chỉ tự ký TLS chưa có Pinning

- **Vị trí:** `IPGS.RemoteControl.CcuClient/RemoteControlClient.cs`

- **Thực trạng:**

Agent tự động sinh chứng chỉ self-signed. Client kết nối qua `SslStream` chấp nhận mọi chứng chỉ hợp lệ về mặt mã hóa nhưng chưa thực hiện Certificate Pinning (so khớp SHA256 fingerprint của agent đã lưu trong profile). Kẻ tấn công trong cùng mạng LAN có thể thực hiện Man-in-the-Middle (MitM) nếu giả mạo được IP/DNS.

2.4 Trải nghiệm & Logic Hiển thị (UI/UX & Monitoring) — Mức độ: TRUNG BÌNH

Vấn đề U1: Đo lường CPU sai lệch trong HealthMonitorWindow

- **Vị trí:** `IPGS.RemoteControl.CcuUI/Views/HealthMonitorWindow.axaml.cs` (Dòng 381–386)
- **Thực trạng:**

```
var match = Regex.Match(cpuLine, @"([\d\.]+)\s*us");
if (match.Success) cpuTxt.Text = $"{match.Groups[1].Value}%";
```

Code chỉ lấy giá trị `us` (User CPU space). Nếu máy ZCU bị nghẽn do I/O (`wa`) hoặc hệ thống (`sy` - kernel capture), tổng tải CPU có thể lên tới 95% nhưng UI chỉ báo 5% (do `us` chỉ chiếm 5%). Cần tính đúng: `Total CPU % = 100 - idle%`.

Vấn đề U2: Cảnh báo Obsolete API Avalonia 12

- **Vị trí:** 32 vị trí trong các tệp giao diện AXAML (`ConnectionEntryWindow`, `FileManagerWindow`, `MountFolderWindow`, `ZcuSetupWizardWindow`…).
- **Thực trạng:** Sử dụng thuộc tính lỗi thời `TextBox.Watermark` thay vì `PlaceholderText`. Cần chuẩn hóa dứt điểm trước khi Avalonia loại bỏ hoàn toàn thuộc tính này trong bản cập nhật lớn tiếp theo.

2.5 Kiểm thử Tự động & Chất lượng Mã nguồn (Testing Gap) — Mức độ: CAO

- **Thực trạng:**

Codebase hiện tại có 3 project nghiệp vụ chính nhưng **hoàn toàn chưa có project Unit Test / Integration Test chuẩn** (`*.Tests.csproj`). Các phần logic cực kỳ nhạy cảm:

- Bộ giải mã gói tin `MessageCodec` (BigEndian parsing, buffer bounds validation).
- Lớp an toàn Shell `ShellQuote` (chống command injection).
- Ánh xạ phím vật lý `PhysicalKeyMapper` và `KeyboardMapper`.
- Bộ tính toán tọa độ `MapToZcuCoords` (letterboxing).

Hiện toàn bộ các thành phần trên chỉ được kiểm tra thủ công qua các file thử nghiệm tạm (`temp_test`, `TestAva`, `TestSsh`), dẫn đến nguy cơ hồi quy (regression) rất cao mỗi khi refactor.

---

3. BẢNG TỔNG HỢP DANH MỤC PHÁT HIỆN

Mã lỗi	Phân hệ	Mức độ	Bản chất vấn đề	Rủi ro phát hành
M1	`CcuUI`	**P1 (Critical)**	Cấp phát 124 MB/s LOH byte[] & WriteableBitmap mỗi frame	Phình RAM, giật đơ UI khi xem lâu
M2	`ZcuAgent`	**P1 (Critical)**	Tạo mới `SKBitmap` unmanaged mỗi frame (8.3 MB malloc/free 15fps)	Phân mảnh heap glibc, tăng VmRSS
M3	`CcuClient`	**P2**	`.ToArray()` sinh rác khi chia chunk upload 64 KB	Tăng GC pressure khi truyền file lớn
C1	`ZcuAgen`	**P1**	Tạo `Task.Run` + `Task.Delay` thừa cho từng	Nghẽn ThreadPool trên máy

Mã lỗi	Phân hệ	Mức độ	Bản chất vấn đề	Rủi ro phát hành
	t	(Critical)	sự kiện chuột	cấu hình yếu
C2	CcuUI	**P1 (Critical)**	Gọi `PasteStatus` từ luồng ThreadPool không qua Dispatcher	Crash app bất thình lình trên client
C3	ZcuAgent	**P2**	`_inputQueue` unbounded không có mouse move coalescing	Trôi chuột, delay điều khiển khi X11 bận
S1	CcuClient	**P1 (Critical)**	Package `SSH.NET` 2025.1.0 dính lỗ hổng bảo mật mức cao	Lỗ hổng kênh truyền quản trị
S2	CcuClient	**P2**	Mật khẩu sudo truyền qua biến môi trường thay vì STDIN	Lộ mật khẩu trong process table Linux
S3	CcuClient	**P2**	Thiếu Certificate Fingerprint Pinning cho kết nối TLS	Nguy cơ MitM trên hạ tầng LAN
U1	CcuUI	**P2**	Tính sai tải CPU (chỉ đọc %us, bỏ qua %sy và %wa)	Báo cáo sai lệch hiệu năng kiosk
U2	CcuUI	**P3**	32 cảnh báo XAML Watermark obsolete	Nợ kỹ thuật giao diện
T1	Toàn bộ	**P1 (Critical)**	Thiếu hoàn toàn bộ test tự động (Unit Test Suite)	Dễ phát sinh lỗi ngầm khi nâng cấp

Báo cáo được chuyển giao sang Tech Lead để xây dựng Phương án Kiến trúc & Đề xuất Cải tiến chi tiết.